```
        left: rect.left + window.scrollX,
      });
    }
  }, [ref]);
  return (
    <div
      style={{
        position: 'absolute',
        ...position,
        background: 'white',
        border: '1px solid blue',
        padding: '5px',
      }}
    >
      {text}
    </div>
  );
};
```

Analysis: This interview question evaluates a candidate's understanding of functional component lifecycles and the differences between `useEffect` and `useLayoutEffect`.

Answer: The issue in this code is that the tooltip's position is calculated inside `useEffect`. Since `useEffect` runs after the browser paints, there is a risk of flickering as the tooltip briefly appears in its initial position before being repositioned.

To address this, `useLayoutEffect` should be used instead. Unlike `useEffect`, it runs synchronously after DOM mutations but before the browser paints. This allows the tooltip's position to be calculated and applied before anything is drawn to the screen, effectively preventing the flickering issue.

Follow-up question (advanced): "Why might flickering not always occur with `useEffect`?"

Answer: Listing 4.7 may not always exhibit this problem because on fast machines or lightweight UIs, the difference between `useEffect` and `useLayoutEffect` can be negligible due to rapid paints. To reliably reproduce the flicker, a slow render can be simulated or artificial delays introduced, as shown in listing 4.8.